

Aula NestJS: Fundamentos de um Backend

Guia de estudo com o registro da construção inicial de uma API com NestJS, detalhando a arquitetura essencial e o ciclo de vida de uma requisição HTTP.

FLUXO DA REQUISIÇÃO: Módulo → Controller → Service → Resposta

O objetivo desta etapa foi entender a estrutura fundamental do framework. O projeto é propositalmente simples: as respostas são textos fixos e não existe banco de dados, DTO, validação ou autenticação. Isso deixa visível o papel de cada peça antes da evolução para uma aplicação completa.

1. Preparando o Projeto

Dentro da pasta do projeto `aula-nest`, instale as dependências padrão:

```
npm install
```

Criando a estrutura pelo Nest CLI

O Nest CLI automatiza a criação dos arquivos e segue os padrões do framework. Para criar um projeto novo do zero, utiliza-se o comando:

```
nest new aula-nest
```

Depois de entrar na pasta do projeto, novos recursos podem ser gerados com `nest generate` (ou a forma abreviada `nest g`):

```
cd aula-nest
nest g module tasks
nest g controller tasks
nest g service tasks
```

Esses comandos criam automaticamente a estrutura do recurso de tarefas:

- `nest g module tasks`: Gera `src/tasks/tasks.module.ts` (define o `TasksModule`).
- `nest g controller tasks`: Gera `src/tasks/tasks.controller.ts` (ponto inicial para rotas).
- `nest g service tasks`: Gera `src/tasks/tasks.service.ts` (concentra as regras de negócio).

O CLI também atualiza os módulos necessários para que os componentes sejam reconhecidos pela aplicação. O `TasksModule` registra o controller e o service, enquanto o `AppModule` importa o módulo de tarefas.

Executar em Desenvolvimento

```
npm run start:dev
```

Inicia o servidor em `http://localhost:3000` com hot-reload ativo.

Outros Comandos Úteis

```
npm run build
```

 (compila)

```
npm run start:prod
```

 (executa build)

2. Como o NestJS Organiza a Aplicação

O NestJS organiza o backend em módulos. Cada módulo representa uma fronteira de funcionalidade e reúne os componentes pertencentes àquele contexto.

Módulos

Um módulo é criado com o decorador `@Module()` e declara `imports`, `controllers` e `providers`. O `AppModule` (em `src/app/app.module.ts`) é o módulo raiz, que registra o `AppController` / `AppService` e importa o `TasksModule`.

Código na prática: `src/tasks/tasks.module.ts`

```
import { Module } from '@nestjs/common';
import { TasksController } from '../tasks.controller';
import { TasksService } from '../tasks.service';

@Module({
  imports: [],
  controllers: [TasksController],
  providers: [TasksService],
})
export class TasksModule {}
```

Entendendo as responsabilidades do arquivo:

1. `import { Module }` : importa o decorator que identifica a classe como módulo NestJS.
2. Imports de `TasksController` e `TasksService` tornam essas classes disponíveis no arquivo.
3. `@Module({ ... })` : objeto de configuração do módulo.
4. `controllers: [TasksController]` : informa quais controllers recebem requisições HTTP deste módulo.
5. `providers: [TasksService]` : registra o service no sistema de injeção de dependência.
6. `imports: []` : utilizado quando o módulo precisa reutilizar recursos de outros módulos.
7. `export class TasksModule {}` : exporta o módulo para ser importado pelo `AppModule`.

No módulo raiz (`src/app/app.module.ts`), a ligação é consolidada assim:

```
import { Module } from '@nestjs/common';
import { AppController } from '../app.controller';
import { AppService } from '../app.service';
import { TasksModule } from '../tasks/tasks.module';

@Module({
  imports: [TasksModule],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

Assim, o NestJS encontra o `TasksModule` a partir do `AppModule` e consegue montar o recurso de tarefas na inicialização.

Controllers

Controllers definem a interface HTTP. O decorador `@Controller()` estabelece o prefixo de rota, e decoradores como `@Get()` e `@Post()` associam métodos a verbos HTTP.

No `AppController`, o prefixo é `/api` (ex: `@Get('/teste')` resulta em `GET /api/teste`). No `TasksController`, o prefixo é `/tasks` (ex: `@Get('/teste')` resulta em `GET /tasks/teste`).

Código na prática: `src/tasks/tasks.controller.ts`

```
import { Controller, Get } from '@nestjs/common';
import { TasksService } from '../tasks.service';

@Controller('tasks')
export class TasksController {
  constructor(private readonly tasksService: TasksService) {}

  @Get()
  getTasks(): string {
    return this.tasksService.getTasks();
  }

  @Get('/teste')
  getTeste(): string {
    return this.tasksService.getHolla();
  }
}
```

O controller não precisa criar o service manualmente com `new TasksService()`. Como `TasksService` está registrado em `providers` dentro do `TasksModule`, o NestJS resolve e injeta essa dependência automaticamente via construtor.

Services e Injeção de Dependência

Services concentram as regras de negócio e a lógica da aplicação. Eles recebem o decorador `@Injectable()` para que o NestJS possa gerenciá-los e injetá-los onde for necessário.

```
// Exemplo de declaração no construtor do Controller:
constructor(private readonly tasksService: TasksService) {}
```

Dessa forma, mantém-se a separação clara de responsabilidades: o **Controller** cuida das questões HTTP (rotas, parâmetros, respostas) e o **Service** cuida das regras funcionais.

3. Inicialização da Aplicação

O arquivo `src/main.ts` é o ponto de entrada da aplicação. A função `bootstrap()` cria a instância da aplicação utilizando `NestFactory.create(AppModule)`, carrega os módulos e inicia o servidor HTTP na porta configurada:

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}

bootstrap();
```

4. Rotas Implementadas

Com a aplicação rodando em `http://localhost:3000`, estas foram as rotas mapeadas na aula:

Método	URL	Responsável	Resposta Emitida
GET	/api	AppController → AppService.getAdeus()	Adeus
GET	/api/teste	AppController	Rota de teste da hello API
POST	/api/teste	AppController	Rota de teste POST da hello API
GET	/tasks	TasksController → TasksService.getTasks()	Lista de tarefas 2
GET	/tasks/teste	TasksController → TasksService.getHolla()	Holla, mundo!

Exemplos de testes via terminal (curl):

```
curl http://localhost:3000/api
curl http://localhost:3000/api/teste
curl -X POST http://localhost:3000/api/teste
curl http://localhost:3000/tasks
curl http://localhost:3000/tasks/teste
```

Nota: O arquivo `api-tests.http` na raiz do projeto permite executar esses mesmos testes de API diretamente pelo VS Code usando extensões como o *REST Client*.

5. Testes E2E (End-to-End)

O arquivo `test/app.e2e-spec.ts` valida o comportamento integrado da API. Ele sobe uma instância do Nest em memória, dispara requisições via `supertest` e valida o status HTTP e a resposta.

```
npm run test:e2e
```

6. Estrutura Final do Projeto

```
aula-nest/
├── src/
│   ├── main.ts                # Ponto de entrada (bootstrap)
│   └── app/
│       ├── app.module.ts      # Módulo raiz
│       ├── app.controller.ts  # Rotas /api
│       └── app.service.ts     # Respostas genéricas
│   └── tasks/
│       ├── tasks.module.ts    # Módulo de tarefas
│       ├── tasks.controller.ts # Rotas /tasks
│       └── tasks.service.ts   # Regras de negócio de tarefas
├── test/
│   └── app.e2e-spec.ts        # Testes de integração HTTP
└── package.json               # Scripts e dependências
```